

Exercises in ISPC Programming

Troels Henriksen
DIKU, University of Copenhagen

November 2025

Task 1: Programming in `ispc`

For these exercises you will be programming in `ispc`. The code handout contains a `Makefile` and benchmarking infrastructure for several programs implemented in both sequential C and `ispc`. Of these, three `ispc` implementations are blank, and it is your task to finish them.

A program `foo` can be benchmarked with the command `make run_foo`. This will also verify that the result produced by the `ispc` implementation matches the result produced by the C implementation. Feel free to change the input generation if you wish, for example to make the input sets larger or smaller.

Generally, do not expect stellar speedups from these programs. A $\times 2$ speedup over sequential C on DIKU's GPU machines should be considered quite good. By default, the `Makefile` sets `ISPC_TARGET=sse4`, which is a relatively old instruction set. You may get better performance by setting it to `host`, which will tell `ispc` to use the newest instruction set supported on the CPU you are using.

All of the subtasks involve some kind of cross-lane communication. Each subtask contains a list of the builtin functions I found useful in my own implementation, but you do not have to use them, and you are welcome to use any others supported by `ispc`. It is likely that my own solution is not optimal anyway.

Subtask 1.1: Prefix sum (`scan.ispc`)

The task here is to implement ordinary *inclusive* prefix sum, which you should be quite familiar with by now.

Recommended builtin functions: `exclusive_scan_add()`, `broadcast()`

Subtask 1.2: Removing neighbouring duplicates (`pack.ispc`)

This program compacts an array by removing duplicate neighbouring elements. It has significant similarities to the filter implementation in `filter.ispc`.

Recommended builtin functions: `exclusive_scan_add()`, `reduce_add()`, `extract()`, `rotate()`

Subtask 1.3: Run-length encoding (`rle.ispc`)

Run-length encoding is a compression technique by which runs of the same symbol (in our case, 32-bit words) are replaced by a *count* and a *symbol*. For example, the C array

```
{ 1, 1, 1, 0, 1, 1, 1, 2, 2, 2, 2 }
```

is replaced by the array

```
{ 3, 1, 1, 0, 3, 1, 4, 2 }
```

Recommended builtin functions: `all()`

Hints: This task is significantly more tricky than the two others. I advise optimising for the case where each symbol is repeated many times, which can be quickly iterated across in a SIMD fashion, falling back to scalar/single-lane execution when a new symbol is encountered. A rough pseudocode skeleton could be:

```
while at least programCount elements remain to be read:
    read next programCount elements
    if all equal to current element:
        increase count and move to next loop iteration
    else:
        # Use single lane to find the mismatch
        if programIndex == 0:
            ...
```